```
# [ { name: 'Honey Badgers', points: 11 },
#   { name: 'Mongooses', points: 8 } ]
```

One place where you see this object destructuring used frequently is at the start of a file in a Node.js program:

```
{pad, trim, dirify} = require 'util'
```

Why? Because the `require` function returns an object, and manually unpacking the object is tedious:

```
util = require 'util'
pad = util.pad
trim = util.trim
dirify = util.dirify
```

Exactly *how* the module system works is covered in detail in chapter 12, but what you can see already is how destructuring is used to remove noise from a program and so helps you to make it simpler.

You're not yet finished with making things simpler, though; CoffeeScript has another bit of syntactic sugar for objects: the object shorthand.

7.2.3. Object shorthand

Consider a `competition` module that exposes a `highlight` and an `initialize` method:

```
makeCompetition = ->
  find = ->
    # function body omitted
  color = ->
    # function body omitted
  highlight = ->
    # function body omitted
  initialize = ->
    # function body omitted

  highlight: highlight,
  initialize: initialize

competition = makeCompetition()
# { highlight: [Function], initialize: [Function] }
```

Expose only the highlight and initialize functions as public by assigning them to properties of the returned object. Remember, you can leave the squigglies (curly braces) off when writing object literals in CoffeeScript.